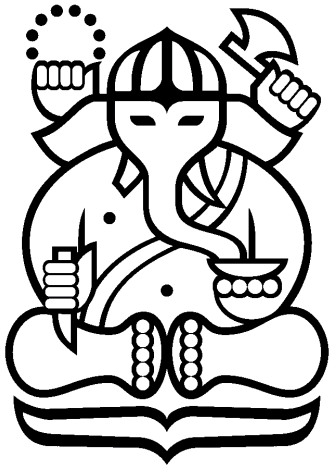
LAPORAN TUGAS 2 IF4020

Kriptografi

Membuat Block Cipher “Baru”



Disusun oleh :

Dave Daniell Yanni	13523003
Richard Christian	13523024
Nathan Jovial Hartono	13523032

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

JL. GANESHA 10, BANDUNG 40132

2026

Daftar Isi

Daftar Isi....2
Pernyataan Tidak Melakukan Kecurangan....3
BAB I
Pendahuluan....4
1.1. Latar Belakang....4
1.2. Review Singkat Block Cipher Sejenis....4
1.3. Gagasan Utama Rancangan Block Cipher....4
1.4. Dasar Teori....5
BAB II
Perancangan Block Cipher....6
2.1. Struktur dan Komponen Algoritma....6
2.2. Penjelasan Alur Sistem....7
BAB III
Implementasi dan Simulasi....10
3.1. Deskripsi Program....10
3.2. Mode Operasi....12
3.3. Bukti Penanganan Padding....16
3.4. Verifikasi Integritas Data....18
BAB IV
Analisis Keamanan....21
4.1. Avalanche Effect....21
4.2. Entropi....22
4.3. Histogram Plaintext dan Ciphertext....23
BAB V
Kesimpulan dan Saran Pengembangan....27
Daftar Pustaka....28
Lampiran....29

Pernyataan Tidak Melakukan Kecurangan

Tugas dan laporan ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri:

 Dave Daniell Yanni (13523003)	 Richard Christian (13523024)
 Nathan Jovial Hartono (13523032)	

BAB I

Pendahuluan

1.1. Latar Belakang

Kriptografi adalah teknik yang digunakan untuk menjaga kerahasiaan data dengan mengubah pesan *plaintext* menjadi pesan tersandi *ciphertext*. Salah satu pendekatan yang dapat digunakan adalah *block cipher*, yaitu algoritma enkripsi simetris yang memproses data dalam blok berukuran tetap menggunakan kunci. Perancangan blok cipher biasanya menggabungkan transformasi data dan pembentukan kunci putaran agar hubungan antara plaintext, kunci, dan ciphertext sulit ditelusuri.

Pada tugas ini, dikembangkan algoritma *block cipher* bernama Venture 128 untuk mempelajari penerapan prinsip-prinsip kriptografi dalam perancangan, implementasi, dan pengujian *block cipher*. Program akan mendukung enkripsi dan dekripsi berkas teks dan biner, dalam mode ECB, CBC, CFB, OFB, dan CTR. Evaluasi hasil algoritma kriptografi akan dianalisis melalui ketepatan hasil dekripsi dan karakteristik *ciphertext* melalui analisis *avalanche effect*, entropi, dan histogram.

1.2. Review Singkat Block Cipher Sejenis

Data Encryption Standard (DES) adalah block cipher yang disetujui oleh *National Bureau of Standards* pada tahun 1976. DES melakukan enkripsi dengan memproses block 64 bit dengan kunci efektif 56 bit melalui 16 putaran Feistel. Struktur ini memungkinkan dekripsi menggunakan fungsi putaran yang sama dengan membalikkan urutan kunci.

Advanced Encryption Standard (AES) adalah block cipher yang ditetapkan oleh *National Institute of Standards and Technology*. Algoritma ini menggunakan block 128 bit, dengan pilihan kunci 128, 192, atau 256 bit. Berbeda dengan DES, AES mengolah state dengan melakukan substitusi byte, menggeser baris, mencampur kolom, dan menambahkan kunci putaran.

1.3. Gagasan Utama Rancangan Block Cipher

Venture128 menggunakan blok dan master key berukuran 128 bit dengan 16 putaran. Block dibagi menjadi 4 word 32 bit, dan diproses menggunakan Type-2 Generalized Feistel Network. Setiap putaran memakai dua kunci untuk mengolah dua cabang, kemudian dilakukan permutasi cabang agar pengaruh dari transformasi menyebar pada putaran selanjutnya.

Fungsi putaran lalu menggabungkan penjumlahan modulo pada dua bagian 16 bit, substitusi S-box, pencampuran berbasis rotasi, AND, dan XOR, serta pencampuran byte

menggunakan matriks M0 atau M1. Sebelum fungsi Feistel diaplikasikan, dilakukan juga permutasi byte dengan menggeser baris ke kanan sesuai indeks putaran.

Master key dikembangkan menjadi 44 word. Dengan 16 putaran, empat word awal dipakai untuk whitening, kemudian 32 word untuk putaran, dan empat word lagi untuk whitening akhir, dengan 4 word tidak digunakan.

1.4. Dasar Teori

Block cipher adalah sejenis kriptografi simetris, yaitu enkripsi dan dekripsi menggunakan kunci yang sama. Algoritma block cipher harus dapat mengembalikan ciphertext menjadi plaintext secara tepat. Block cipher biasanya memiliki iterasi dimana transformasi diulang dalam beberapa putaran dengan kunci yang diturunkan dari master key. Struktur fungsi Feistel memungkinkan proses dibalik tanpa membalikkan fungsi putaran.

Mode operasi mengatur penggunaan *block cipher* untuk data yang terdiri atas banyak blok. ECB, operasi yang paling sederhana memproses blok secara terpisah. Mode CBC mengaitkan blok dengan *ciphertext* sebelumnya. Mode CFB menggunakan *feedback ciphertext*, sedangkan OFB menggunakan feedback *output* enkripsi. CTR mengenkripsi nilai *counter* yang bertambah, yang memungkinkan operasi paralel.

Efektivitas dari algoritma cipher yang diimplementasikan lalu dievaluasi menggunakan *Avalanche,* Entropi, dan Histogram. *Avalanche* mengukur persentase bit ciphertext yang berubah ketika satu bit pada *plaintext* atau kunci diubah. Entropi mengukur keacakan distribusi byte. Histogram memperlihatkan frekuensi kemunculan setiap nilai byte.

BAB II

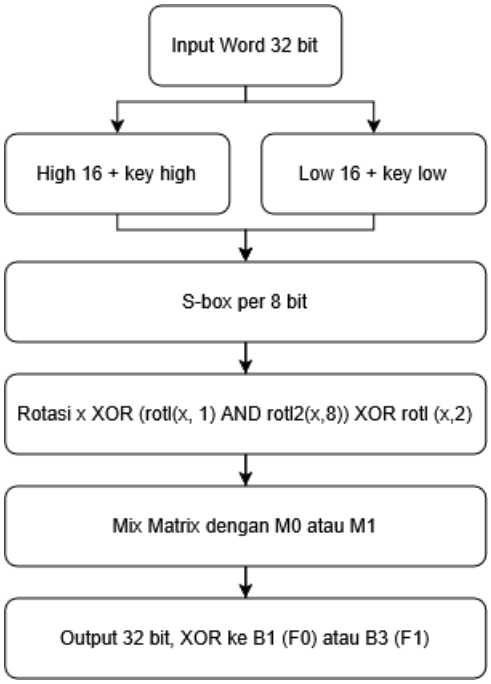
Perancangan Block Cipher

2.1. Struktur dan Komponen Algoritma

Venture128 dirancang sebagai Type-2 Generalized Feistel Network dengan 4 cabang. Setiap blok akan dibagi menjadi state B0, B1, B2, B3, dengan masing-masing cabang sebagai word berukuran 32 bit. Berikut adalah parameter utama rancangan algoritma:

Parameter	Nilai
Ukuran Blok	128 bit
Panjang Master Key	128 bit
Jumlah Cabang	4 cabang 32 bit
Jumlah Putaran	16
Kunci Setiap Putaran	2 word 32 bit
Whitening	XOR dengan 4 word sebelum dan sesudah

Dalam setiap putaran, dua cabang akan menjadi masukan fungsi F, dimana F0 akan menerima B0, lalu hasil fungsi ini akan di-XOR ke B1. F1 lalu menerima B2 dan kunci lainnya, dan melakukan XOR ke B3. Urutan pemasukan cabang kemudian dipermutasi agar operasi tersebar. F0 dan F1 masing-masing akan menerima word dan kunci 32 bit, dan menghasilkan word 32 bit. Keduanya memiliki urutan operasi yang sama, dengan matriks pencampuran yang berbeda, yaitu M0 untuk F0 dan M1 untuk F1.



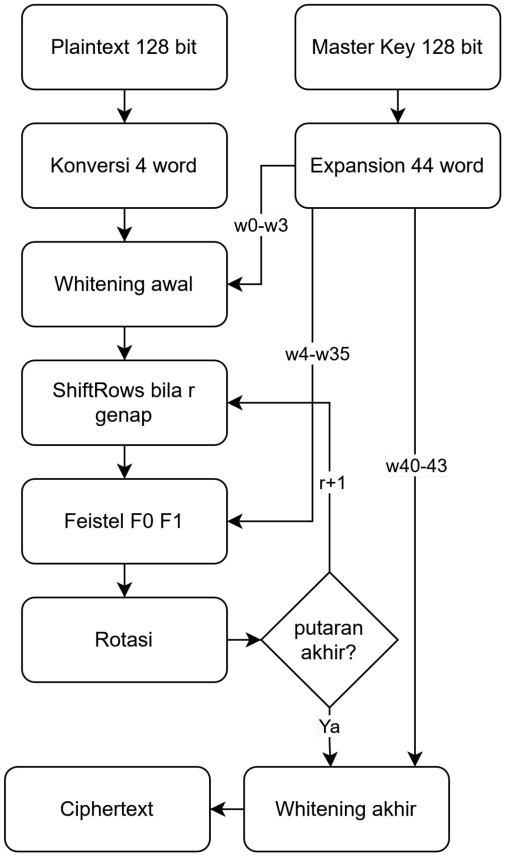
Gambar 2.1 Ilustrasi Fungsi F0/F1 Venture128

Berikut merupakan ilustrasi proses yang dilakukan oleh fungsi F0/F1 pada word yang diterima. Proses dimulai dengan memecah word menjadi 2 bagian lalu melakukan penjumlahan dengan key yang diterima dengan mod 2^16. Hasil penjumlahan ini lalu digabungkan dan dilakukan substitusi dengan S-box 8 bit, menghasilkan 4 byte yang kemudian digabungkan lagi menjadi satu word.

Pemrosesan lalu dilanjutkan dengan percampuran setiap word yang menggunakan rotasi, AND, dan XOR.. Hasil percampuran rotasi ini lalu dicampur lagi menggunakan matriks M0 atau M1, yang mencampurkan keempat byte menjadi empat byte baru. Hasil percampuran matriks ini menjadi hasil output dari fungsi F0/F1, yang akan digunakan untuk melakukan XOR ke block selanjutnya.

2.2. Penjelasan Alur Sistem

Berikut merupakan diagram yang menjelaskan alur enkripsi satu blok:



Gambar 2.2 Alur enkripsi satu blok Venture 128

Alur Venture128 dimulai dari blok data dan kunci, yang dilanjutkan dengan pengolahan state 16 putaran, hingga blok keluaran. Data yang diproses dalam satu siklus adalah 16 byte. Master key yang diterima diekspansi menjadi 44 word, yang kemudian akan dipakai untuk whitening dan masukan fungsi. Enkripsi diawali dengan membagi plaintext menjadi 4 word 32 bit, yang akan melalui tahap whitening awal menggunakan hasil ekspansi master key w0-w3 dengan operasi XOR. Hasil whitening lalu dilanjutkan ke dalam putaran.

Pada proses putaran, setiap iterasi diberi indeks r dari 0 hingga 15. Pada setiap indeks genap, state akan mengalami operasi ShiftRows ke kanan. Baris pertama tidak bergeser, sedangkan baris berikutnya bergeser sesuai urutan baris masing-masing, satu, dua, dan tiga kolom. Pada indeks ganjil, tidak dilakukan operasi ShiftRows. Setelah proses ini, word lalu dimasukan ke dalam fungsi F0/F1, yang menerima 2 word hasil ekspansi untuk melakukan operasi. Dengan total 16 putaran, 32 word hasil ekspansi akan digunakan, yaitu w4-w35. Setelah proses putaran selesai, dilakukan kembali proses whitening menggunakan w40-w43. Hasil whitening ini kemudian digabungkan kembali menjadi ciphertext 16 byte.

Proses dekripsi melakukan operasi serupa, diawali dengan melakukan ekspansi kunci. Dekripsi lalu dilanjutkan dengan membalikan hasil whitening dengan melakukan operasi XOR pada ciphertext, kemudian melakukan putaran yang diproses dari indeks r 15 hingga 0. Setelah melalui seluruh putaran yang dibalik, hasil whitening awal juga dibalikan hingga akhirnya state dikonversi kembali menjadi plaintext.

BAB III

Implementasi dan Simulasi

3.1. Deskripsi Program

Program diimplementasikan berbasis CLI menggunakan modul argparse Python dengan struktur modular.

- main.py: Pengelola parsing CLI, routing mode operasi, pembacaan masukan (string, file, hex, bit, stdin), dan penulisan luaran.
- block_modes/: Implementasi matematis 5 mode operasi (ecb.py, cbc.py, cfb.py, ofb.py, ctr.py).
- modules/enc_dec.py: Fungsi enkripsi/dekripsi 1 blok 128-bit Venture128 berbasis Type-2 GFN 16 putaran.
- modules/padding.py: Penanganan padding dan unpadding sesuai standar PKCS#7.
- modules/key_expand.py: Pembangkitan round keys (16 subkunci putaran).
- modules/iv.py: Pembangkitan cryptographically secure random IV via secrets.token_bytes dan parsing IV kustom.

```
def main() -> None:
    parser = build_parser()
    args = parser.parse_args()

    try:
        master_key = read_key(args)
        round_keys = generate_round_keys(master_key)
        input_data = read_input_data(args)
        mode = args.block_mode
        iv = None
        if mode in ("cbc", "cfb", "ofb", "ctr"):
            if args.iv:
                iv = parse_iv(args.iv[0], args.iv[1])
            elif args.enc_dec_mode == "encrypt":
                iv = generate_iv()
            else:
                if len(input_data) < BLOCK_SIZE_BYTES:
                    raise ValueError(
                        f"ciphertext is too short to contain a {BLOCK_SIZE_BYTES}-byte IV"
                    )
                iv = input_data[:BLOCK_SIZE_BYTES]
                input_data = input_data[BLOCK_SIZE_BYTES:]
        if mode == "ecb":
            if args.iv:
                print("venture128: warning: ignoring -iv flag for ECB", file=sys.stderr)
            res = (
                ecb.ecb_encrypt(input_data, round_keys)
                if args.enc_dec_mode == "encrypt"
```

```
            else ecb.ecb_decrypt(input_data, round_keys)
        )
    elif mode == "cbc":
        res = (
            cbc.cbc_encrypt(input_data, round_keys, iv)
            if args.enc_dec_mode == "encrypt"
            else cbc.cbc_decrypt(input_data, round_keys, iv)
        )
    elif mode == "cfb":
        res = (
            cfb.cfb_encrypt(input_data, round_keys, iv)
            if args.enc_dec_mode == "encrypt"
            else cfb.cfb_decrypt(input_data, round_keys, iv)
        )
    elif mode == "ofb":
        res = (
            ofb.ofb_encrypt(input_data, round_keys, iv)
            if args.enc_dec_mode == "encrypt"
            else ofb.ofb_decrypt(input_data, round_keys, iv)
        )
    elif mode == "ctr":
        res = (
            ctr.ctr_encrypt(input_data, round_keys, iv)
            if args.enc_dec_mode == "encrypt"
            else ctr.ctr_decrypt(input_data, round_keys, iv)
        )

    result_data = res[0] if isinstance(res, tuple) else res
    if args.histogram:
        plaintext, ciphertext = (
            (input_data, result_data) if args.enc_dec_mode == "encrypt"
            else (result_data, input_data)
        )
        source_name = Path(args.file).name if args.file else "input"
        chart_path = Path("histograms") / f"{Path(source_name).stem}_histogram.png"
        protected = [Path(path).resolve() for path in
                     (args.file, args.output, args.key_file) if path]
        if args.iv and args.iv[0].lower() == "file":
            protected.append(Path(args.iv[1]).resolve())
        if chart_path.resolve() in protected:
            raise ValueError("Histogram PNG path conflicts with an input or output file.")
        ciphertexts = compare_modes(plaintext, round_keys, mode, ciphertext, iv)
        write_histogram_png(chart_path, plaintext, ciphertexts, source_name)

    # Handle tuple return (data, iv) from encrypt modes
    if isinstance(res, tuple):
        output_data, used_iv = res
        if not args.iv and args.enc_dec_mode == "encrypt":
            output_data = used_iv + output_data
    else:
```

```
            output_data = res

        write_output(output_data, args)

    except (ValueError, OSError) as e:
        parser.exit(1, f"venture128: error: {e}\n")
```

Alur kerja program diawali dengan membaca dan mengonversi masukan dari berbagai format (file, string, hex, atau stdin) menjadi raw bytes, bersamaan dengan pemvalidasian kunci 16 byte untuk membangkitkan deretan round keys. Pada mode yang membutuhkan vector inisialisasi (CBC, CFB, OFB, dan CTR), program menyiapkan IV 16 byte secara input manual atau dibangkitkan otomatis, lalu menyisipkan di awal ciphertext, sebelum mengeksekusi proses enkripsi atau dekripsi sesuai mode blok yang dipilih. Terakhir, hasil pemrosesan dikonversi ke format keluaran yang ditentukan (raw, hex, atau bits) untuk ditulis ke file atau stdout, dengan opsi analisis visual distribusi byte berupa grafik histogram PNG.

3.2. Mode Operasi

Terdapat 5 mode, ECB, CBC, CFB, OFB, dan CTR. Akan dilakukan pengujian dengan input dibawah

Key	Text	Biner
0123456789abcdef	Kriptografi tugas 2 block cipher venture128 gg	000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f202122232425262728292a2b2c2d2e2f30313233343536373839 3a3b3c3d3e3f404142434445464748494a4b4c4d4e4f5051525354555657585 95a5b5c5d5e5f606162636465666768696a6b6c6d6e6f707172737475767778797a7b7c7d7e7f808182838485868788898a8b8c8d8e8f909192939495969798999a9b9c9d9e9fa0a1a2a3a4a5a6a7a8a9aaabacadaeafb0b1b2b3b4b5b6b7b8b9babbbcbdbebfc0c1c2c3c4c5c6c7c8c9cacbcccdcecfd0d1d2d3d4d5d6d7d8d9dadbdcdddedfe0e1e2e3e4e5e6e7e8e9eaebecedeeeff0f1f2f3f4f5f

		6f7f8f9fafbfcfdfeff

venture128 --block-mode {mode} -e -key "0123456789abcdef" -f {filepath}
Contoh:

```
venture128 --block-mode ecb -e -key "0123456789abcdef" -f
./tests/input/sample_text.txt
d823b0533be60927a8b861b686042798b4d84fa52bd7830ce84a767adef3fc0a75e2ecdcd
df2d4e885ea88f4a8502328
```

Mode	Output Encrypt Text	Output Encrypt Biner
ECB	d823b0533be60927a8b861b686042798b4d84fa52bd7830ce84a767adef3fc0a75e2ecdcddf2d4e885ea88f4a8502328	c3536bd4823c8601b244488f75458bb4228043a055b8540482e14381ed7000a4c507462e465a2e3e77951418522d38290682b6f437f107e8214230ac54c60153e6fb8d3518dee27b000317f6d87d87233c0c61ababee4f0928abadbf65a18eb49040587ecee1e77ff25ef6793fec4e4003a7eba266a10e5c133c68e2c380a8f0e40851f65c59c4b4c5b8f18dc99b3815570860dabe4a333a83d10b37f9340467464a0f4f2dc7f186e9a2f08c56319f0bbe74eb44dba3c066f6c5b02731683ccf7ec180f7180fe7c73bf58d8db4a9ee8475e90353a67e496b1e2f97f253a18af4dc0c7b19f6d7fe2ba1869e72f38f0db3c008d07276bc77ed9f95e6aa72493cc352458948b5aabcd9b25bb0c457abec8f
CBC	ea8c31d49808b7c5bfee3d1353acd5cc4bfc145feae99aa9a4fe2a651db531b2f808bf57e4b8b613f614b4438071feb6728f6fb88fbf6e442b7e26dd1db3eb37	85498ab11e2631a75379eed1fe8f098c27af3da0bf5f76e56730f5085b012934ebc003f39b781a74a316b6e04d1c2ca5e6be95e5bde6e00fbbaaa5aac788fa77cdba2e6fe320dac0d2fa9b1e6e9ed116ea6d4faf456507d25659183297421d037fcbf3d78a571b27d4bd26f3833f5a527b685

		2d41f8d6efaea1b4398b5a87e285ea454c0ed33e2e955fe6783148062d0ae2621f7a33deff391e4ba80d6ef9e7499770ca76d2f681fe34e7acd7f963fcb589043164ef336e3978dd83bc5d1234c0f1f7f0c951e60f82a5b313c7cc054eb46a49e6eef7782f477d2ffee8ecdbd8a18f5abc3b09d0812df30e506d71b9e925627773e774cfbb331bba160e5bfb751197f7c57f210f2b1c49ef3c401b5b435d4c37f470669241ff6d1f71dc092f34e
CFB	50b34da972a60f4f4f1806f7909422a4fe145ea49b72ec4034972e10d758450d8e8288da3266c655ba644206b420ef9e7fde69a4dbe6fb224a1cf61d58e3	f3dfa1630d64a9488b356428ecb68d90aec6216f58d3ca3328b935a647c4a38d9e3a40d513e6857b49493adf34742b71c4c1dcae9f3b22e1c6c3ad1c55b636fdfd51829925aa5e1458811435afb42acad3f62c7f157064b5a86905e013dd1b70e36151a46e64f260a024dcebeb35a084773d2efb96373011e6a2db756f5a1884a40c7e801c2aaf9f50960180cf57d00e665482ab33833734ad44f2abdf363581e230b403e4b81070af64f9b8c4a20fc50fc814c5201044f01b185a0b51b025d1c720822f913438f496ed26707d637968b44af0726b9e20202fbf56cb3f596eb9a200600f912cbe2188a422da758acf74af0bd1943c57db992a4480af355cc72b95a7c49f8af33aa1b6b3bc4c57ecdfeb3
OFB	44eddf79873cf561c15dba032d9f88f0eee76c05d04c0138a49f6ad2c8d25cf8b25857996a67f68fb41ce274645d042a4090c728ddf17ccbe8a6c00f5d434dda	dff0cc5abef12e5bc4af1536974962d07f31bc327071b4f233134c1112875c7177fbd6e065869b0f6977d32232f96944c05c65d5efd4969e8bd08969c7938ef6fb73d6392852e42ffab5f6a7afccaae1b9fca2ffd143c84b06e171c7c

		0de58189f6e375eddc993058a66ff20f471294038b0263c87f1e4cf08dd81243b601bfb92402d35550dbed11f706b8e9f7f9dd7bfb842cf59d0b03c4b15892e70b2d4a69f19c1f0c8fca28877dd5562bafec6fc210f512dcfedd5cb085c1a7d0885c15a77ed7ba774d32892bdf75e721252375368e20839c72331f0eafec0066b79b17835147922eb0a521c0ea84b73354ae131c96983939054292d07b99cbc60c8c0e29235a7c0d4e011ec36c741112f68e4a9ce7d51e313bc260d5c8a71bd3c058d6e
CTR	b43304d7dae302cda8dd8dd09df174d15c1ba53d27308085c638e357435f70772c929b8ebf57b19e0e737b3662d7bb81d5741946a9e99ef498b932ee617089fe	582039704577ce5614fa62b8bcbcbdf28cb20f401a1a9c0f7c88a0e332dc59e68863f1dabd760fb23d9c291509815cac62ca3bf300357e556f1638f9ff770ac05466a8c8410c824106b3fa92e097b82e51ef730573ee0d6107bbb75677d015b52de0ca85c7dd8524d32205a56a40ae8ef8a5c16620f63d015553f45812d57d01612952151dce31410c95622d710cdd93451810168e920f5be47c49d7be77a7789c3f837f3f7c30c88b3fd6b8e58ad58a22aa7bd5df78f26accfcba45d3709afbe28c902edb4e2eb19f55cb07f31ce19f7694cb6bc27a4e043dc2765079006a7e3b1dc18b75aede533a4a551fa9364a268dc122b8198aa1671087fa5e5aabf7aa606014a003d2ff284e7c3f4f78e47cd55f2698b7a249b5b4215bffa2410b5e30

venture128 --block-mode ctr -d -key "0123456789abcdef" -hex “ciphertext”, -of raw
ditambahkan untuk konversi hex menjadi karakter
Contoh:

```
venture128 --block-mode ecb -d -key "0123456789abcdef" -hex
"d823b0533be60927a8b861b686042798b4d84fa52bd7830ce84a767adef3fc0a75e2ecdc
ddf2d4e885ea88f4a8502328" -of raw
Kriptografi tugas 2 block cipher venture128 gg
```

Mode	Output Decrypt Text	Output Decrypt Biner
ECB	Kriptografi tugas 2 block cipher venture128 gg	000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f20212222324252627282 92a2b2c2d2e2f30313233343536373839 3a3b3c3d3e3f404142434445464748494a4b4c4d4e4f5051525354555657585 95a5b5c5d5e5f606162636465666768696a6b6c6d6e6f707172737475767778797a7b7c7d7e7f808182838485868788898a8b8c8d8e8f909192939495969798999a9b9c9d9e9fa0a1a2a3a4a5a6a7a8a9aaabacadaeafb0b1b2b3b4b5b6b7b8b9babbbcbdbebfc0c1c2c3c4c5c6c7c8c9cacbcccdcecfd0d1d2d3d4d5d6d7d8d9dadbdcdddedfe0e1e2e3e4e5e6e7e8e9eaebecedeeeff0f1f2f3f4f5f6f7f8f9fafbfcfdfeff
CBC	Sama seperti atas	Sama seperti atas
CFB	Sama seperti atas	Sama seperti atas
OFB	Sama seperti atas	Sama seperti atas
CTR	Sama seperti atas	Sama seperti atas

3.3. Bukti Penanganan Padding

```
BLOCK_SIZE_BITS = 128
BLOCK_SIZE_BYTES = BLOCK_SIZE_BITS // 8

from venture128.variables.constants import BLOCK_SIZE_BYTES
```

```
def pkcs7_pad(data: bytes, block_size: int = BLOCK_SIZE_BYTES) -> bytes:
    n = block_size - (len(data) % block_size)
    return data + bytes([n]) * n


def pkcs7_unpad(data: bytes, block_size: int = BLOCK_SIZE_BYTES) -> bytes:
    if not data or len(data) % block_size:
        raise ValueError("Invalid PKCS#7 length")
    n = data[-1]
    if n < 1 or n > block_size or data[-n:] != bytes([n]) * n:
        raise ValueError("Invalid PKCS#7 padding")
    return data[:-n]
```

Skema padding yang digunakan adalah PKCS#7 dengan ukuran blok B = 16 byte. Jika panjang data menyisihkan n = B - (len(data) mod(B)) maka akan ditambahkan n byte bernilai n ke akhir datanya.

Dapat terjadi 2 kasus

1. Panjang data bukan kelipatan ukuran blok
   Contoh data 11 byte, maka n = 5
   Data asli : FF FF FF FF FF FF FF FF FF FF FF
   Data dengan padding: FF FF FF FF FF FF FF FF FF FF FF 05 05 05 05 05
2. Panjang data tepat kelipatan ukuran blok
   Contoh data 16 byte, maka akan menambah 1 blok penuh berukuran 16 byte
   Data : FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
   Padding : 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10

Contoh pemanggilan:

```
from venture128.modules.padding import *
from venture128.modules.enc_dec import encrypt_block, decrypt_block
from venture128.variables.constants import BLOCK_SIZE_BYTES

def ecb_encrypt(data: bytes, keys) -> bytes:
    data = pkcs7_pad(data)
    out = bytearray()
    for i in range(0, len(data), BLOCK_SIZE_BYTES):
        out += encrypt_block(data[i:i + BLOCK_SIZE_BYTES], keys)
    return bytes(out)

def ecb_decrypt(data: bytes, keys) -> bytes:
    if not data or len(data) % BLOCK_SIZE_BYTES:
        raise ValueError(f"Ciphertext length is not multiple of {BLOCK_SIZE_BYTES}
bytes.")
    out = bytearray()
    for i in range(0, len(data), BLOCK_SIZE_BYTES):
        out += decrypt_block(data[i:i + BLOCK_SIZE_BYTES], keys)
    return pkcs7_unpad(bytes(out))
```

Semua mode lain juga memanggil fungsi padding.

3.4. Verifikasi Integritas Data

Untuk verifikasi integritas data dilakukan perbandingan hash.

```
import hashlib
import sys
from pathlib import Path

REPO_ROOT = Path(__file__).resolve().parents[1]
SRC_DIR = REPO_ROOT / "src"
if str(SRC_DIR) not in sys.path:
    sys.path.insert(0, str(SRC_DIR))

from venture128.block_modes import cbc, cfb, ctr, ecb, ofb
from venture128.modules.key_expand import generate_round_keys

KEY = b"0123456789abcdef"
IV = bytes.fromhex("000102030405060708090a0b0c0d0e0f")
MODES = ["ecb", "cbc", "cfb", "ofb", "ctr"]


def run_encrypt(mode: str, data: bytes, round_keys: list, iv: bytes) -> bytes:
    if mode == "ecb":
        res = ecb.ecb_encrypt(data, round_keys)
    elif mode == "cbc":
        res = cbc.cbc_encrypt(data, round_keys, iv)
    elif mode == "cfb":
        res = cfb.cfb_encrypt(data, round_keys, iv)
    elif mode == "ofb":
        res = ofb.ofb_encrypt(data, round_keys, iv)
    elif mode == "ctr":
        res = ctr.ctr_encrypt(data, round_keys, iv)
    return res[0] if isinstance(res, tuple) else res


def run_decrypt(mode: str, data: bytes, round_keys: list, iv: bytes) -> bytes:
    if mode == "ecb":
        res = ecb.ecb_decrypt(data, round_keys)
    elif mode == "cbc":
        res = cbc.cbc_decrypt(data, round_keys, iv)
    elif mode == "cfb":
        res = cfb.cfb_decrypt(data, round_keys, iv)
    elif mode == "ofb":
        res = ofb.ofb_decrypt(data, round_keys, iv)
    elif mode == "ctr":
        res = ctr.ctr_decrypt(data, round_keys, iv)
    return res[0] if isinstance(res, tuple) else res


def test_sha256_integrity(round_keys: list) -> None:
    plaintext = b"Testing ventrure 128 data integrity."
```

```
    pt_hash = hashlib.sha256(plaintext).hexdigest()
    print(f"Original Plaintext SHA-256 : {pt_hash}\n")

    for mode in MODES:
        ct = run_encrypt(mode, plaintext, round_keys, IV)
        dec = run_decrypt(mode, ct, round_keys, IV)
        dec_hash = hashlib.sha256(dec).hexdigest()
        is_match = pt_hash == dec_hash
        status = "MATCH" if is_match else "FAILED"
        print(f"[{mode.upper():<3}] Decrypted SHA-256: {dec_hash} | {status}")


def main():
    round_keys = generate_round_keys(KEY)
    test_sha256_integrity(round_keys)


if __name__ == "__main__":
    main()
```

```
python ./tests/test_integrity.py
Original Plaintext SHA-256 :
52655bf247f4add77ea80d0a626fd079d180dab15219e80fcf1326297f736279

[ECB] Decrypted SHA-256:
52655bf247f4add77ea80d0a626fd079d180dab15219e80fcf1326297f736279 | MATCH
[CBC] Decrypted SHA-256:
52655bf247f4add77ea80d0a626fd079d180dab15219e80fcf1326297f736279 | MATCH
[CFB] Decrypted SHA-256:
52655bf247f4add77ea80d0a626fd079d180dab15219e80fcf1326297f736279 | MATCH
[OFB] Decrypted SHA-256:
52655bf247f4add77ea80d0a626fd079d180dab15219e80fcf1326297f736279 | MATCH
[CTR] Decrypted SHA-256:
52655bf247f4add77ea80d0a626fd079d180dab15219e80fcf1326297f736279 | MATCH
```

Berdasarkan pengujian integritas data menggunakan fungsi cryptographic hash SHA-256, diperoleh nilai hash digest dari plaintext awal sebesar 52655bf247f4add77ea80d0a626fd079d180dab15219e80fcf1326297f736279. Setelah melalui enkripsi dan dekripsi pada semua mode, seluruh data hasil dekripsi menghasilkan nilai SHA-256 yang persis identik dengan plaintext asalnya. Hasil ini membuktikan secara matematis bahwa implementasi algoritma Venture128 bersifat lossless, invertible, dan deterministik di seluruh mode operasi, serta menjamin bahwa proses transformasi biner berlangsung sempurna tanpa adanya korupsi data atau distorsi byte.

BAB IV

Analisis Keamanan

4.1. Avalanche Effect

Jalankan test test_avalanche.py

```
python ./tests/test_avalanche.py
key       : 30313233343536373839616263646566 (b'0123456789abcdef')
IV        : 000102030405060708090a0b0c0d0e0f
================================================================
========
plaintext_1block.txt (16 bytes)
mode  test       trials    avg%    min%    max%   ct_len
ecb   plaintext     128    24.80    18.36    30.47      32
ecb   key           128    50.20    44.53    57.03      32
cbc   plaintext     128    49.93    42.97    58.98      32
cbc   key           128    49.91    42.97    57.81      32
cfb   plaintext     128     0.78     0.78     0.78      16
cfb   key           128    49.33    35.16    59.38      16
ofb   plaintext     128     0.39     0.39     0.39      32
ofb   key           128    49.44    40.23    58.20      32
ctr   plaintext     128     0.39     0.39     0.39      32
ctr   key           128    49.74    41.41    58.20      32

================================================================
========
plaintext_2block.txt (32 bytes)
mode  test       trials    avg%    min%    max%   ct_len
ecb   plaintext     128    16.49    12.76    20.31      48
ecb   key           128    50.11    44.27    57.81      48
cbc   plaintext     128    41.56    27.60    54.17      48
cbc   key           128    49.78    44.27    56.25      48
cfb   plaintext     128    12.79     0.39    32.81      32
cfb   key           128    49.61    39.84    58.20      32
ofb   plaintext     128     0.26     0.26     0.26      48
ofb   key           128    49.62    43.75    56.77      48
ctr   plaintext     128     0.26     0.26     0.26      48
ctr   key           128    49.79    44.27    56.77      48

================================================================
========
plaintext_long.txt (4320 bytes)
mode  test       trials    avg%    min%    max%   ct_len
ecb   plaintext     128     0.18     0.15     0.22    4336
ecb   key           128    49.63    43.95    54.84    4336
cbc   plaintext     128    25.37     0.71    50.46    4336
cbc   key           128    49.98    49.20    50.68    4336
cfb   plaintext     128    25.11     0.41    50.24    4320
```

```
cfb   key          128    50.00    49.42    50.61     4320
ofb   plaintext    128     0.00     0.00     0.00     4336
ofb   key          128    49.99    49.26    50.68     4336
ctr   plaintext    128     0.00     0.00     0.00     4336
ctr   key          128    49.99    49.39    50.95     4336
```

Hasil pengujian Avalanche Effect pada file berukuran besar (plaintext_long.txt, 4,320 byte) menunjukkan bahwa perubahan 1 bit pada kunci secara konsisten menghasilkan perubahan bit ciphertext rata-rata sebesar 50% untuk seluruh mode operasi. Hal ini membuktikan bahwa tercapai difusi global yang ideal.

Sebaliknya, Plaintext Avalanche mencerminkan karakteristik arsitektur masing-masing mode secara presisi: mode stream (OFB dan CTR) menghasilkan persentase perubahan mendekati 0% karena sifat bit-for-bit mapping dari keystream, mode ECB menghasilkan 0.18% karena isolasi antar-blok (efek perubahan terisolasi pada 1 blok terkait), sedangkan mode berantai (CBC dan CFB) menghasilkan rata-rata 25% akibat propagasi kesalahan searah dari titik perubahan bit hingga akhir file.

Di sisi lain, pengujian Avalanche Effect pada sampel kecil seperti plaintext_1block.txt (16 byte) dan plaintext_2block.txt (32 byte) unreliable untuk penarikan kesimpulan statistik. Pada sampel 1 hingga 2 blok, persentase perubahan plaintext sangat dipengaruhi oleh bias posisi bit yang diubah. Sebagai contoh, pada mode CBC, mengubah bit di blok pertama akan merusak seluruh blok setelahnya, sekitar 50%, sedangkan mengubah bit di blok terakhir hanya merusak blok itu sendiri sekitar 25%. Oleh karena itu, angka rata-rata pada sampel kecil berfluktuasi secara ekstrem dan hanya pengujian pada file berukuran besar yang dapat memberikan gambaran difusi statistik yang valid.

4.2. Entropi

Jalankan test test_entropy.py

```
python ./tests/test_entropy.py
ENTROPY TEST
Key       : 30313233343536373839616263646566 (b'0123456789abcdef')
IV        : 000102030405060708090a0b0c0d0e0f
================================================================
==================
Input: plaintext_1block.txt (16 bytes)
Plaintext Entropy : 4.0000 / 8.0000 bits/byte (50.00% of max 8.0, sample upper bound
= 4.0000)
Mode   Plain(B) Cipher(B)  H(Plain)  H(Cipher)  % of Max
ECB         16       32    4.0000     4.8125    60.16%
CBC         16       32    4.0000     4.8750    60.94%
CFB         16       16    4.0000     3.7500    46.88%
OFB         16       32    4.0000     4.8750    60.94%
CTR         16       32    4.0000     4.6639    58.30%
```

```
================================================================
==================
Input: plaintext_2block.txt (32 bytes)
Plaintext Entropy : 4.0000 / 8.0000 bits/byte (50.00% of max 8.0, sample upper bound
= 5.0000)
Mode   Plain(B) Cipher(B)  H(Plain)  H(Cipher)  % of Max
ECB         32       48    4.0000     4.7202    59.00%
CBC         32       48    4.0000     5.2359    65.45%
CFB         32       32    4.0000     4.8750    60.94%
OFB         32       48    4.0000     5.3766    67.21%
CTR         32       48    4.0000     5.3766    67.21%

================================================================
==================
Input: plaintext_long.txt (4320 bytes)
Plaintext Entropy : 3.6587 / 8.0000 bits/byte (45.73% of max 8.0, sample upper bound
= 8.0000)
Mode   Plain(B) Cipher(B)  H(Plain)  H(Cipher)  % of Max
ECB        4320     4336    3.6587     5.8494    73.12%
CBC        4320     4336    3.6587     7.9522    99.40%
CFB        4320     4320    3.6587     7.9521    99.40%
OFB        4320     4336    3.6587     7.9600    99.50%
CTR        4320     4336    3.6587     7.9611    99.51%
```

Pengujian entropi pada file plaintext_long.txt (4,320 byte) membuktikan kemampuan algoritma dalam mengacak data terstruktur (H_Plain = 3.6587 bits/byte). Mode operasi chain dan stream (CBC, CFB, OFB, dan CTR) berhasil mengacak plaintext hingga mencapai tingkat entropi mendekati batas maksimum teoritis (H_Cipher sekitar 7.95-7.96 bits/byte, atau sekitar 99.4%-99.5% dari nilai maksimum 8.0). Nilai yang mendekati 8.0 ini menunjukkan bahwa distribusi byte ciphertext sangat seragam dan secara statistik tidak dapat dibedakan dari pseudo random noise. Sebaliknya, mode ECB hanya mencatatkan entropi sebesar 5.8494 bits/byte (73.12%), yang mengonfirmasi kelemahan utama ECB dalam membocorkan pola struktur berulang pada data masukan.

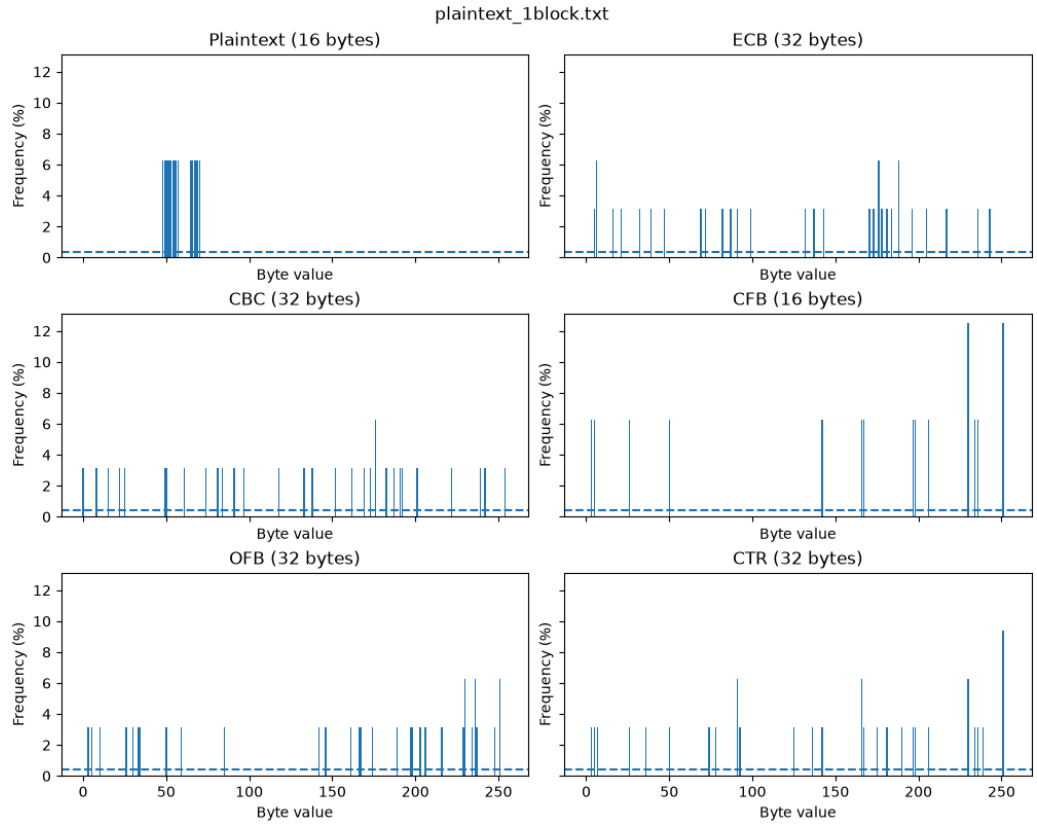
Sama halnya dengan avalanche effect, skor entropi pada file 1-blok dan 2-blok tidak dapat diandalkan untuk mengukur performa kriptografi secara obyektif. Hal ini disebabkan oleh keterbatasan teoritis ukuran sampel, di mana batas atas entropi untuk data sepanjang N byte dibatasi secara matematis oleh log2(N). Sampel 16 byte hanya memiliki nilai entropi maksimum sebesar 4.0000 bits/byte, sedangkan sampel 32 byte sebesar 5.0000 bits/byte. Oleh sebab itu, persentase entropi yang terlihat rendah pada sampel kecil (sekitar 46%-67%) merupakan batasan statistik dari jumlah byte yang diuji, bukan mencerminkan kualitas pengacakan algoritma.

4.3. Histogram Plaintext dan Ciphertext

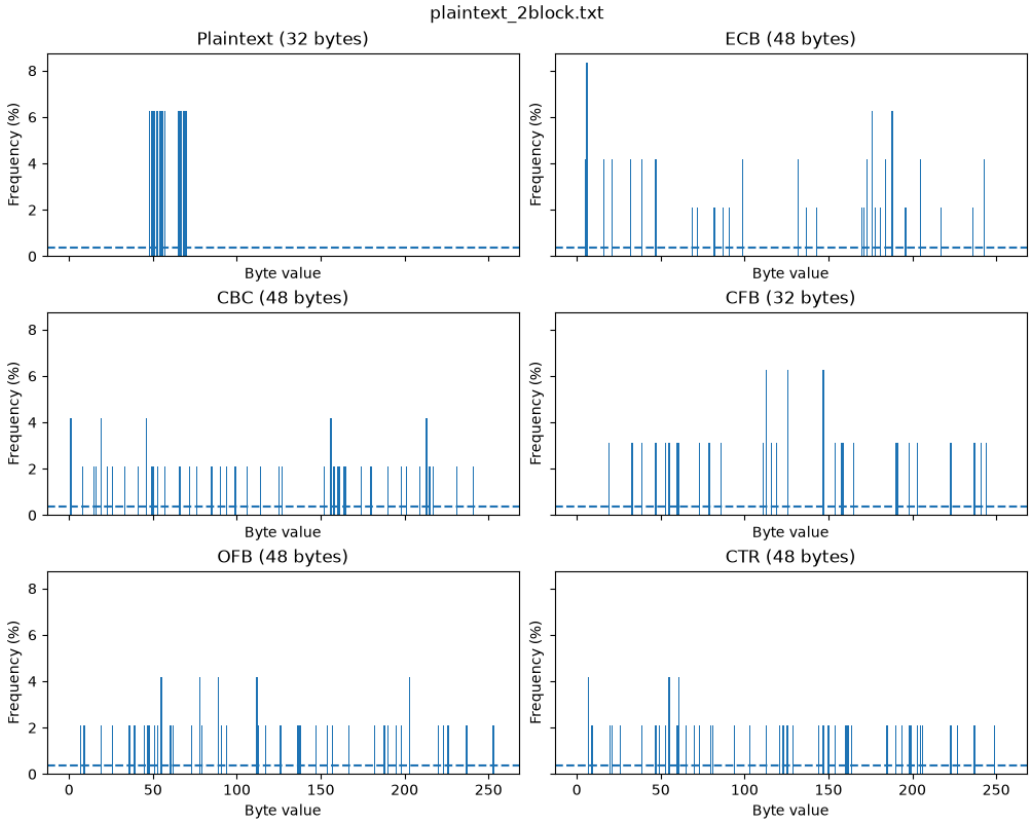
Analisis histogram dilakukan dengan menjalankan perintah dengan opsi --histogram. Pengujian histogram dilakukan pada tiga berkas masukan, yaitu plaintext 1 block, 2 block, dan teks hasil salinan spek. Perintah dengan opsi --histogram akan secara otomatis

membandingkan hasil plaintext dengan mode ECB, CBC, CFB, OFB, dan CTR sekaligus. Berikut merupakan hasil pengujian:

cipher_1block.txt
uv run venture128 --block-mode ctr -e -key-file tests/input/key_default.txt -f tests/input/plaintext_1block.txt --histogram -o cipher_1block_ctr.bin

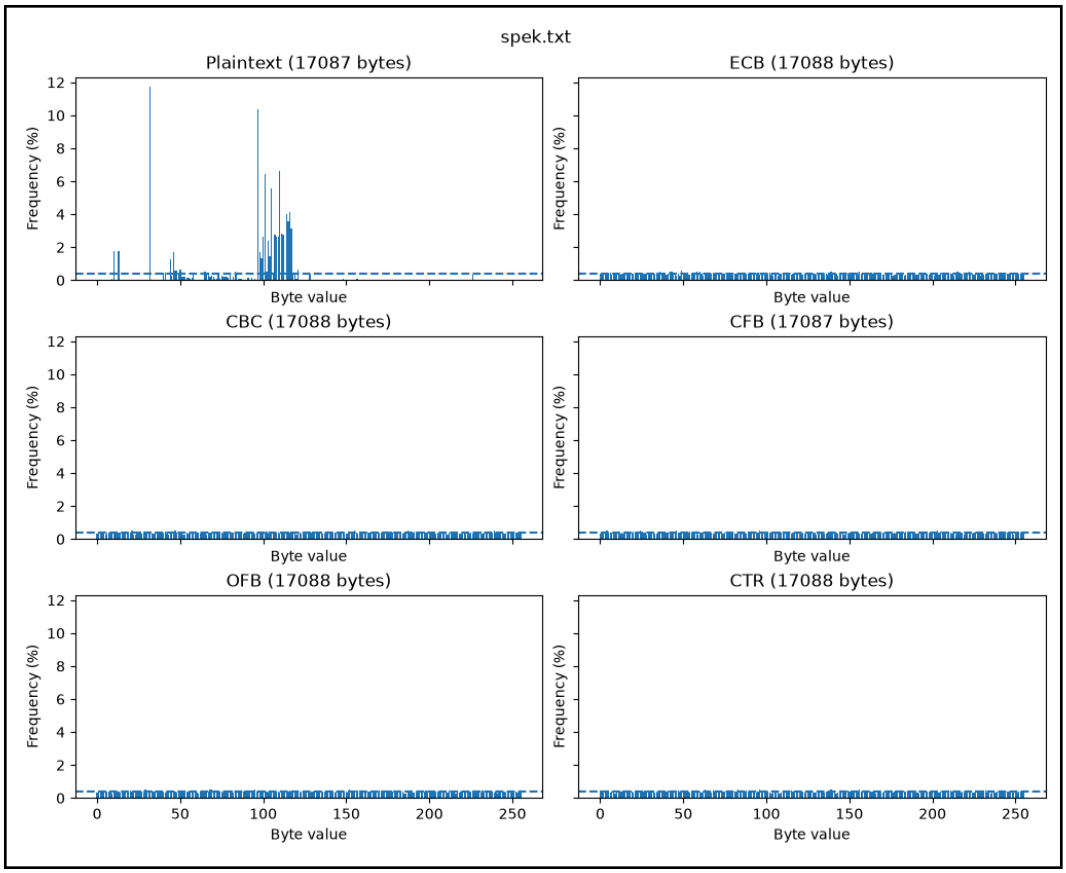


cipher_2block.txt
uv run venture128 --block-mode ctr -e -key-file tests/input/key_default.txt -f tests/input/plaintext_2block.txt --histogram -o cipher_1block_ctr.bin



spek.txt

uv run venture128 --block-mode ctr -e -key-file tests/input/key_default.txt -f tests/input/spek.txt --histogram -o cipher_spek_ctr.bin



Analisis histogram dilakukan dengan menghitung jumlah kemunculan byte asli. Dalam perhitungan, IV/counter tidak dimasukkan dalam perhitungan, sedangkan padding tetap dihitung. Garis putus-putus yang berada pada grafik menggambarkan nilai “ideal” dimana setiap byte terdistribusi secara merata.

Pada input satu block, plaintext hanya menggunakan byte angka dan huruf A-F, sehingga setiap nilai muncul dengan frekuensi 6.25%. Karena jumlah byte yang dienkripsi sedikit, distribusi histogram yang dihasilkan masih kurang merata.

Input dua blok tidak jauh berbeda, karena jumlah byte tidak bertambah banyak dari 1 blok. Walaupun begitu, sudah terlihat sedikit peningkatan dari distribusi byte dibandingkan masukan 1 blok. Setiap mode menghasilkan distribusi yang cukup merata, dengan pengecualian pada CFB, dimana karena hasil hanya 16 byte pada 1 block dan 32 byte pada 2 block, maka distribusi juga terbatas oleh jumlah byte.

Input spek.txt memberikan sampel yang lebih besar dengan ukuran file 17KB, dengan isi teks yang lebih realistik. Histogram *plaintext* terlihat mempunyai distribusi yang terpusat pada beberapa byte, tetapi hasil enkripsi dengan Venture128 berhasil memberikan keluaran yang sangat merata, apapun mode yang digunakan.

BAB V

Kesimpulan dan Saran Pengembangan

Venture128 berhasil dirancang dan diimplementasikan sebagai block cipher berbasis Type-2 Generalized Feistel Network dengan ukuran blok dan master key 128 bit, 16 putaran, serta whitening awal dan akhir. Program mendukung lima mode operasi (ECB, CBC, CFB, OFB, dan CTR) beserta padding PKCS#7, dan seluruh pengujian menunjukkan hasil dekripsi identik dengan plaintext asli. Verifikasi menggunakan SHA-256 menghasilkan nilai hash yang sama pada semua mode, sehingga implementasi terbukti lossless dan invertible.

Dari sisi keamanan, perubahan satu bit kunci menghasilkan perubahan sekitar 50% bit ciphertext pada semua mode, yang menunjukkan difusi yang baik. Entropi ciphertext pada berkas besar mencapai sekitar 7,95 bits/byte (±99,4%–99,5%) untuk mode CBC, CFB, OFB, dan CTR, dan histogram ciphertext pada spek.txt terlihat sangat merata. Mode ECB masih memperlihatkan kelemahan karena pola data dapat bocor (entropi 5,8494 bits/byte). Pengujian pada sampel 1 dan 2 blok tidak dapat dijadikan dasar kesimpulan karena keterbatasan jumlah byte.

Pengembangan selanjutnya dapat berfokus pada analisis kriptanalisis yang lebih mendalam, seperti uji linear dan diferensial terhadap S-box dan matriks M0/M1, serta uji keacakan statistik formal. Selain itu, dapat ditambahkan dukungan ukuran kunci 192 dan 256 bit, mekanisme autentikasi seperti MAC atau mode AEAD (misalnya GCM) untuk menjamin integritas ciphertext, serta optimasi performa agar pemrosesan berkas berukuran besar lebih cepat.

Daftar Pustaka

Katagi, M., & Moriai, S. (2011). *The 128-bit blockcipher CLEFIA* (RFC 6114). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc6114

Munir, R. (2026a). *08 – Block cipher (Bagian 1)* [Bahan kuliah IF4020 Kriptografi]. Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung. Diakses pada 13 Oktober 2026, dari https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi/2026-2027/08-Block-Cipher-Bagian1-(2026).pdf

Munir, R. (2026b). *09 – Review beberapa block cipher (Bagian 1: DES dan Triple DES)* [Bahan kuliah IF4020 Kriptografi]. Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung. Diakses pada 13 Oktober 2026, dari https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi/2026-2027/09-Beberapa-block-cipher-bagian1-2026.pdf

Munir, R. (2026c). *10 – Review beberapa block cipher (Bagian 2: Advanced Encryption Standard – AES)* [Bahan kuliah IF4020 Kriptografi]. Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung. Diakses pada 13 Oktober 2026, dari https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi/2026-2027/10-Beberapa-block-cipher-bagian2-2026.pdf

Munir, R. (2026d). *11 – Review beberapa block cipher (Bagian 3: GOST, RC5, RC6, Serpent, dll)* [Bahan kuliah IF4020 Kriptografi]. Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung. Diakses pada 13 Oktober 2026, dari https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi/2026-2027/11-Beberapa-block-cipher-bagian3-2026.pdf

Beaulieu, R., Shors, D., Smith, J., Treatman-Clark, S., Weeks, B., & Wingers, L. (2013). *The SIMON and SPECK families of lightweight block ciphers* (Cryptology ePrint Archive, Paper 2013/404). International Association for Cryptologic Research. https://eprint.iacr.org/2013/404

Lampiran

Github: https://github.com/19623248Git/IF4020_Tugas2_2026_2027

Pembagian tugas:

Dave Daniell Yanni 13523003	Semua
Richard Christian 13523024	Semua
Nathan Jovial Hartono 13523032	Semua